Build runbook

The plan tab says what to achieve and why. This tab is the keyboard-level sequence: do these in order, and do not start a step until the one before it is verified.

Tonight — steps 1 to 3

No code. Rule 1 forbids commits before the day. These three steps only put bytes on disk and knowledge in your head.

Step 1 — prove the toolchain on every laptop. Run these four and fix anything that fails:

```
docker --version
docker compose version        # must print v2.x
python3.11 --version
git --version
```

Then check free disk with `df -h` — you need 14 GB clear — and note each laptop's RAM. Designate the one with the most RAM as the demo host and write that down.

Step 2 — pull the images. This is the step that saves your Checkpoint 0.

```
git clone https://github.com/open-telemetry/opentelemetry-demo.git
cd opentelemetry-demo
make start
```

It will pull roughly 14 GB. Let it finish, then `docker compose ps` and confirm every container is up. Leave the images on disk — do not prune Docker before the event.

Step 3 — watch one fault end to end, with your own eyes. Fifteen minutes, and it is the difference between understanding the demo and reading about it.

1. Open `http://localhost:8080/` and click around the store so there is traffic.
2. Open `http://localhost:8080/feature`, set `paymentFailure` to 100%, save.
3. Wait two minutes. Open `http://localhost:8080/jaeger/ui/`, pick service `checkout`, and find a red trace. Expand it and look for the deepest span that is red — it will be in `payment`. **This is the single most important thing to see before the event.** The alert will name `frontend` or `checkout`; the broken thing is `payment`.
4. Open `http://localhost:9090` and run `traces_span_metrics_calls_total` in the expression box. Look at the label names it actually gives you and write them down.
5. Turn the flag back off and confirm the errors stop.

Then `make stop` (or `docker compose down`) so you are not leaving 6 GB resident overnight. Do **not** delete the images.

12:00–13:00 — steps 4 to 6

Step 4 (12:00–12:10) — fork and everyone commits. Do this before touching anything else.

```
# one person forks the template repo on GitHub, then everyone:
git clone <your-team-fork>
cd <repo>
# each member edits the team form or README, adds their own name, then:
git add -A && git commit -m "add <your name>" && git push
```

Verify on GitHub that every member shows in the commit list. Checkpoint 0 checks this and it is free points you can lose by forgetting.

Step 5 (12:10–12:40) — stack up, in parallel with step 6. On the demo host:

```
cd opentelemetry-demo
make start
docker compose ps          # every container Up
```

Then discover the two ephemeral ports and write them on the whiteboard:

```
docker compose port opensearch 9200
docker compose port jaeger 16686
```

Step 6 (12:10–12:55) — verify every data source by hand, with curl. No Python yet. Each of these must return real data before anyone writes a tool. If one fails, you have found a problem worth an hour of your afternoon, now instead of at 14:00.

```
# 1. services exist
curl -s "http://localhost:8080/jaeger/ui/api/services"

# 2. traces come back
curl -s "http://localhost:8080/jaeger/ui/api/traces?
service=frontend&lookback=15m&limit=2"

# 3. prometheus is alive and has the span metrics
curl -s "http://localhost:9090/api/v1/label/__name__/values" | tr ',' '\n' |
grep -i -E 'span|container'

# 4. the error-ratio query returns rows
curl -s "http://localhost:9090/api/v1/query" --data-urlencode \
  'query=sum by (service_name) (rate(traces_span_metrics_calls_total[5m]))'

# 5. logs are indexed (OS_PORT from step 5)
curl -s "http://localhost:$OS_PORT/_cat/indices?v" | grep otel-logs
curl -s "http://localhost:$OS_PORT/otel-logs-*/_search?size=2" -H 'Content-
Type: application/json' \
  -d '{"query":{"match_all":{}}}'
```

Read the output of #5 carefully and write down the real field names. The OpenSearch document shape is the one thing in this runbook most likely to differ from what you expect. Find what holds the message text, what holds the service name, what holds the timestamp, and what holds the severity. Your `search_logs` tool is built from those four names.

Meanwhile, two people do these in parallel: one writes the LLM smoke test (a single tool called `get_time` returning a fixed string — prove the model calls it and uses the result), one pastes the organisers' schema into `schema.json` and writes a hand-made example that validates. If the LLM smoke test fails, that is the only blocker that cannot be worked around — escalate to a mentor immediately.

Step 7 (13:00) — agree the file layout

Write this on the whiteboard before anyone opens an editor. Five files, clear owners, no overlap, so two people never edit the same file.

File	Owner	Holds
`config.py`	Host	The four base URLs and the discovered ports. Nothing else. Every other file imports from here.

File	Owner	Holds
`tools.py`	Tools	The six telemetry functions and the evidence registry. No LLM imports anywhere in this file.
`agent.py`	Loop	The system prompt, the tool schemas for the model, the loop, the CLI entry point.
`report.py`	Output	Schema validation and turning a diagnosis into readable text.
`run_all.py`	Evidence	Loops over the practice incidents and writes the scoring table.

`config.py` exists so that when a port changes at 16:00 — and it will, if anyone restarts the stack — exactly one line changes:

```
# config.py — the only place URLs are written down
JAEGER_BASE_URL = "http://localhost:8080/jaeger/ui"
PROMETHEUS_BASE_URL = "http://localhost:9090"
OPENSEARCH_BASE_URL = "http://localhost:32773"   # from: docker compose port
opensearch 9200
DEFAULT_WINDOW_MINUTES = 15
MAX_ITEMS_PER_TOOL_RESULT = 15
```

The rule that matters: `tools.py` never imports the LLM SDK, and `agent.py` never calls `requests` directly. Keeping that wall means you can debug a broken tool without spending an LLM call, and you will do that fifty times today.

13:00–14:00 — steps 8 to 10, the tools

Build these in order. Each one is finished only when you have run it from the shell against a **live injected incident** and read the output with your eyes.

Step 8 — `get_service_health`. This is the agent's first move on every incident, so build it first.

```
import requests
import config

def run_prometheus_query(promql_query_string):
    # one place where Prometheus is called, so there is one place to debug
    response = requests.get(
        config.PROMETHEUS_BASE_URL + "/api/v1/query",
        params={"query": promql_query_string},
        timeout=15,
```

```
    )
    response.raise_for_status()
    parsed_body = response.json()
    # Prometheus wraps everything in {"status": ..., "data": {"result":
[...]}}
    return parsed_body["data"]["result"]


def get_service_health(window_minutes=5):
    error_ratio_query = (
        "sum by (service_name) "
        "(rate(traces_span_metrics_calls_total"
        '{status_code="STATUS_CODE_ERROR"}[' + str(window_minutes) + "m])) "
        "/ sum by (service_name) "
        "(rate(traces_span_metrics_calls_total[" + str(window_minutes) +
"m]))"
    )
    latency_p95_query = (
        "histogram_quantile(0.95, sum by (service_name, le) "
        "(rate(traces_span_metrics_duration_milliseconds_bucket["
        + str(window_minutes) + "m])))"
    )

    # start with an empty dictionary keyed by service name, then fill it
twice
    health_by_service_name = {}

    for result_row in run_prometheus_query(error_ratio_query):
        service_name = result_row["metric"].get("service_name", "unknown")
        # result_row["value"] is a two-item list: [unix_timestamp, "the value
as a string"]
        error_ratio_as_float = float(result_row["value"][1])
        health_by_service_name[service_name] = {
            "service": service_name,
            "error_ratio": round(error_ratio_as_float, 4),
            "p95_latency_ms": None,
        }

    for result_row in run_prometheus_query(latency_p95_query):
        service_name = result_row["metric"].get("service_name", "unknown")
        latency_as_float = float(result_row["value"][1])
        if service_name not in health_by_service_name:
            health_by_service_name[service_name] = {
                "service": service_name,
                "error_ratio": 0.0,
                "p95_latency_ms": None,
```

```
            }
        health_by_service_name[service_name]["p95_latency_ms"] =
round(latency_as_float, 1)

    # hand back a plain list, worst error ratio first, so the model reads the
    # most suspicious service in the first line instead of hunting for it
    all_rows = list(health_by_service_name.values())
    all_rows.sort(key=sort_key_worst_error_first, reverse=True)
    return all_rows


def sort_key_worst_error_first(one_row):
    # a named function instead of a lambda, so the sort is readable and
testable
    return one_row["error_ratio"]
```

The Python worth noticing here: `.get("service_name", "unknown")` reads a dictionary key with a fallback instead of raising `KeyError` when a series has no such label — in telemetry, labels go missing and you do not want a crash. `result_row["value"][1]` picks the second item of a two-item list, because Prometheus returns `[timestamp, value]` and the value arrives as a string, which is why `float()` wraps it. And `sort(key=...)` takes a function that is handed one row and returns the thing to sort on; passing the function name without brackets passes the function itself rather than calling it.

Step 9 — `search_traces` and `get_trace`. `get_trace` is the most important function you will write today, because flattening the span tree correctly is what lets the model find the deepest failing span.

```
def get_trace(trace_id):
    response = requests.get(
        config.JAEGER_BASE_URL + "/api/traces/" + trace_id, timeout=15
    )
    response.raise_for_status()
    # Jaeger returns {"data": [ one_trace ]} even for a single trace
    raw_trace = response.json()["data"][0]

    # Jaeger gives process IDs, not service names, in each span.
    # Build a lookup first: {"p1": "checkout", "p2": "payment", ...}
    service_name_by_process_id = {}
    for process_id in raw_trace["processes"]:
        service_name_by_process_id[process_id] = raw_trace["processes"]
[process_id]["serviceName"]

    flattened_spans = []
    for raw_span in raw_trace["spans"]:
```

```
        parent_span_id = None
        for reference in raw_span.get("references", []):
            if reference["refType"] == "CHILD_OF":
                parent_span_id = reference["spanID"]

        span_has_error = False
        exception_message = None
        for tag in raw_span.get("tags", []):
            if tag["key"] == "error" and tag["value"] is True:
                span_has_error = True
            if tag["key"] == "otel.status_code" and tag["value"] == "ERROR":
                span_has_error = True
            if tag["key"] in ("exception.message", "error.message"):
                exception_message = str(tag["value"])[:300]

        flattened_spans.append({
            "span_id": raw_span["spanID"],
            "parent_span_id": parent_span_id,
            "service": service_name_by_process_id[raw_span["processID"]],
            "operation": raw_span["operationName"],
            "duration_ms": round(raw_span["duration"] / 1000.0, 1),
            "has_error": span_has_error,
            "exception": exception_message,
        })

    return {
        "trace_id": trace_id,
        "span_count": len(flattened_spans),
        "spans": flattened_spans,
        "deepest_failing_services":
find_deepest_failing_services(flattened_spans),
    }


def find_deepest_failing_services(flattened_spans):
    # a failing span whose own children did not fail is where the problem
starts;
    # everything above it is just propagating the error upward
    span_ids_that_have_a_failing_child = set()
    for span in flattened_spans:
        if span["has_error"] and span["parent_span_id"] is not None:
            span_ids_that_have_a_failing_child.add(span["parent_span_id"])

    deepest_services = []
    for span in flattened_spans:
        if span["has_error"] and span["span_id"] not in
```

```
span_ids_that_have_a_failing_child:
            deepest_services.append({
                "service": span["service"],
                "operation": span["operation"],
                "exception": span["exception"],
            })
    return deepest_services
```

`set()` is doing real work in that last function: you need to ask "does this span have any failing child?" once per span, and a set answers `in` instantly where a list would scan every item. `raw_span["duration"] / 1000.0` converts microseconds to milliseconds — Jaeger reports microseconds and forgetting this makes every latency look a thousand times worse. `str(tag["value"])[:300]` is slicing: take the first 300 characters, because a Java stack trace in a prompt is money burned.

Computing `deepest_failing_services` in Python rather than asking the model to work it out is deliberate. It is a deterministic rule, so a `for` loop gets it right every time and an LLM gets it right most of the time.

Step 10 — `search_logs`, `get_container_health`, `search_traces`, `list_services`. Same pattern. For each one: call the API, pull out four or five fields, cap the list at `config.MAX_ITEMS_PER_TOOL_RESULT`, truncate long strings, return a plain list of dictionaries. Build `search_logs` from the field names you wrote down in step 6 — filter by service and time window, never query the whole index.

Before moving on, run every tool from the shell against a live fault. Add this to the bottom of `tools.py`:

```
if __name__ == "__main__":
    import sys, json
    function_name = sys.argv[1]
    function_arguments = sys.argv[2:]
    function_to_call = globals()[function_name]
    print(json.dumps(function_to_call(*function_arguments), indent=2))
```

Then `python tools.py get_service_health` and `python tools.py get_trace <some-trace-id>`. `globals()` hands you the module's top-level names as a dictionary, so `globals()["get_trace"]` is the function itself, and `*function_arguments` unpacks a list into separate arguments. Twelve lines, and it means you never need an LLM call to test a tool.

14:00–15:30 — steps 11 to 13, the agent

Step 11 — the loop. Every provider's tool-calling API has the same shape: you send messages plus a list of tool descriptions, the model replies either with text or with a request to call a tool, you execute it and append the result, and you go round again.

```
def run_investigation(alert_text, maximum_iterations=10):
    conversation_messages = [{"role": "user", "content": alert_text}]
    investigation_log = []          # every tool call and result, for the
report

    for iteration_number in range(maximum_iterations):
        model_response = call_the_model(
            system_prompt=SYSTEM_PROMPT,
            messages=conversation_messages,
            tools=TOOL_DESCRIPTIONS,
        )
        conversation_messages.append(model_response.as_message())

        requested_tool_calls = model_response.tool_calls
        if len(requested_tool_calls) == 0:
            # the model stopped asking for tools, so it is done
            break

        for tool_call in requested_tool_calls:
            tool_function = AVAILABLE_TOOLS[tool_call.name]
            try:
                tool_result = tool_function(**tool_call.arguments)
            except Exception as error:
                # never let a broken tool kill the run; tell the model
instead
                tool_result = {"error": str(error)}

            investigation_log.append({
                "iteration": iteration_number,
                "tool": tool_call.name,
                "arguments": tool_call.arguments,
                "result": tool_result,
            })
            conversation_messages.append(
                format_tool_result_message(tool_call.id, tool_result)
            )

    return conversation_messages, investigation_log
```

The Python worth noticing: `range(maximum_iterations)` is the loop cap that stops a confused model calling tools forever and eating your spend limit. `**tool_call.arguments` unpacks a dictionary into named arguments, so `{"service": "cart"}` becomes `service="cart"` — which is why your tool parameter names must exactly match the names in the tool descriptions you give the model. The `try` / `except` around the tool call is not optional: a tool that raises takes down the whole run, where a tool that returns `{"error": ...}` lets the model try something else and often still get the right answer.

Step 12 — the system prompt. Keep it in its own file so you can diff versions. It needs five things, and the third is where your points are:

1. Who it is: an on-call engineer investigating one incident in a microservice store.
2. The tools it has, one line each.
3. **The investigation cascade from the plan tab, written as rules.** Metrics first. Then, if there are errors, traces — and *the failing component is the deepest erroring span whose own children did not error, not the service the alert named*. Then logs for that service at that timestamp. If there are no errors but there is latency, check container health instead.
4. The evidence rule: cite only reference tokens that tools actually returned, never invent a trace ID or a log line.
5. The confidence rule: high only when two of the three sources agree on one service; if no source names a specific component, escalate rather than guess.

Add one line that pays for itself: *call one tool at a time and decide the next from what you just learned*. The brief explicitly rewards this over pasting everything into one prompt.

Step 13 (15:00–15:30) — rehearse the live run. Three full dry runs with a stopwatch:

```
<faultctl inject some-incident>
sleep 120
python agent.py "error rate elevated on frontend"
```

Decide who drives the keyboard and who narrates. Have the terminal open with the command typed and not entered. Know what you will say if it fails — "it found the right service but the log tool timed out, here is the transcript" is a fine answer and honesty is scored; silence and frantic typing is not.

15:30–18:30 — steps 14 to 17, the diagnosis

Step 14 — make the final answer a tool call. Add a seventh tool that the model calls to finish. Its arguments are the schema fields, so the output is structured before you ever see it:

```
def submit_diagnosis(root_cause_service, root_cause_summary,
                     confidence, evidence_tokens, proposed_action):
    invented_tokens = check_cited_evidence(evidence_tokens)
    if len(invented_tokens) > 0:
        # hand it straight back to the model instead of accepting it
        return {
            "accepted": False,
            "reason": "These evidence references were not returned by any
tool "
                      "in this run: " + ", ".join(invented_tokens)
                      + ". Resubmit citing only references you actually
received.",
        }
    diagnosis = {
        "root_cause_service": root_cause_service,
        "root_cause_summary": root_cause_summary,
        "confidence": confidence,
        "evidence": [],
        "proposed_action": proposed_action,
    }
    for token in evidence_tokens:
        diagnosis["evidence"].append(evidence_registry[token])
    return {"accepted": True, "diagnosis": diagnosis}
```

When `accepted` is `True`, your loop stops. When it is `False`, the message goes back into the conversation and the model tries again. `", ".join(invented_tokens)` turns a list into one comma-separated string — `join` is called on the separator, not on the list, which reads backwards until you have written it a few times.

Then validate `diagnosis` against `schema.json` with `jsonschema.validate()`. If it raises, print which field failed and fix the tool description so the model cannot produce that shape again. Do not patch the output by hand.

Step 15 — wire the evidence registry into every tool. Each tool, before returning, registers each item and attaches its token:

```
def search_logs(service=None, contains=None, window_minutes=15, limit=15):
    raw_log_records = query_opensearch(service, contains, window_minutes,
limit)
    returned_items = []
    for record in raw_log_records:
        one_item = {
            "timestamp": record["time"],
            "service": record["service_name"],
            "severity": record["severity_text"],
            "message": str(record["body"])[:300],
        }
        one_item["reference"] = register_evidence_item("log", one_item)
        returned_items.append(one_item)
    return returned_items
```

Every item the model sees now carries a `reference` it can cite, and nothing else validates.

Step 16 (16:15–17:30) — the sweep. `run_all.py` loops over all ten practice incidents: inject, sleep 120, run the agent, record what it said, reset. Run it once, go get coffee, come back to a table.

```
| incident | expected | agent said | evidence real | tool calls | pass |
```

Do not fix anything until all ten have run. Then look at the table and fix the failure mode that appears most often — one prompt change usually moves three incidents at once. Re-run the sweep after each change and keep the old table so you can see whether you improved things or just moved the failures around.

Step 17 (17:30–18:00) — calibration. Add the confidence rule to the prompt, re-run the sweep, and check one number: of the incidents it got wrong, how many did it report as high confidence? Every one of those is worse than an escalation. Push until that number is zero, even if it means the agent escalates on two incidents it could have guessed right — the brief says plainly that an escalation with findings scores better than a confident wrong fix.

18:00–18:45 — step 18, freeze

No new features after 18:00. In order:

1. `docker compose down && make start`, wait for green, then one clean run from scratch. If it works from nothing, it will work in front of a mentor.
2. Everyone commits and pushes. Check the GitHub contributor list, not your own memory of who typed what.
3. `git tag day1-end && git push --tags`.

4. Write `evaluation-note.md` from the scoring table: which incidents it got right, which it got wrong, and why. Ten minutes now, an hour tomorrow.
5. Copy the system prompt into the repo as its own file.
6. Five lines at the top of the README: what works, what does not, what to fix first tomorrow.
7. Agree out loud who owns the Action API wrapper, the approval gate and the recovery check tomorrow, before anyone leaves the room.

The one-sentence test of whether Day 1 succeeded: someone who has never seen your repo can clone it, run one command against a freshly injected incident, and get back schema-valid JSON naming the right service with evidence that actually exists. If that is true at 18:30, Day 2 is mostly plumbing.